

elements in the set are central, and so the probability that our random choice of splitter produces a central element is $\frac{1}{2}$. Hence, by our simple waiting-time bound (13.7), the expected number of iterations before a central element is found is 2; and so the expected number of iterations spent in phase j , for any j , is at most 2.

This is pretty much all we need for the analysis. Let X be a random variable equal to the number of steps taken by the algorithm. We can write it as the sum $X = X_0 + X_1 + X_2 + \cdots$, where X_j is the expected number of steps spent by the algorithm in phase j . When the algorithm is in phase j , the set has size at most $n(\frac{3}{4})^j$, and so the number of steps required for one iteration in phase j is at most $cn(\frac{3}{4})^j$ for some constant c . We have just argued that the expected number of iterations spent in phase j is at most two, and hence we have $E[X_j] \leq 2cn(\frac{3}{4})^j$. Thus we can bound the total expected running time using linearity of expectation,

$$E[X] = \sum_j E[X_j] \leq \sum_j 2cn \left(\frac{3}{4}\right)^j = 2cn \sum_j \left(\frac{3}{4}\right)^j \leq 8cn,$$

since the sum $\sum_j (\frac{3}{4})^j$ is a geometric series that converges. Thus we have the following desired result.

(13.18) *The expected running time of `Select`(n, k) is $O(n)$.*

A Second Application: Quicksort

The randomized divide-and-conquer technique we used to find the median is also the basis of the sorting algorithm `Quicksort`. As before, we choose a splitter for the input set S , and separate S into the elements below the splitter value and those above it. The difference is that, rather than looking for the median on just one side of the splitter, we sort both sides recursively and glue the two sorted pieces together (with the splitter in between) to produce the overall output. Also, we need to explicitly include a base case for the recursive code: we only use recursion on sets of size at least 4. A complete description of `Quicksort` is as follows.

```

Quicksort(S):
  If |S| ≤ 3 then
    Sort S
    Output the sorted list
  Else
    Choose a splitter  $a_i \in S$  uniformly at random
    For each element  $a_j$  of S

```

```

        Put  $a_j$  in  $S^-$  if  $a_j < a_i$ 
        Put  $a_j$  in  $S^+$  if  $a_j > a_i$ 
    Endfor
    Recursively call Quicksort( $S^-$ ) and Quicksort( $S^+$ )
    Output the sorted set  $S^-$ , then  $a_i$ , then the sorted set  $S^+$ 
Endif

```

As with median-finding, the worst-case running time of this method is not so good. If we always select the smallest element as a splitter, then the running time $T(n)$ on n -element sets satisfies the same recurrence as before: $T(n) \leq T(n-1) + cn$, and so we end up with a time bound of $T(n) = \Theta(n^2)$. In fact, this is the worst-case running time for Quicksort.

On the positive side, if the splitters selected happened to be the medians of the sets at each iteration, then we get the recurrence $T(n) \leq 2T(n/2) + cn$, which arose frequently in the divide-and-conquer analyses of Chapter 5; the running time in this lucky case is $O(n \log n)$.

Here we are concerned with the *expected running time*; we will show that this can be bounded by $O(n \log n)$, almost as good as in the best case when the splitters are perfectly centered. Our analysis of Quicksort will closely follow the analysis of median-finding. Just as in the `Select` procedure that we used for median-finding, the crucial definition is that of a *central splitter*—one that divides the set so that each side contains at least a quarter of the elements. (As we discussed earlier, it is enough for the analysis that each side contains at least some fixed constant fraction of the elements; the use of a quarter here is chosen for convenience.) The idea is that a random choice is likely to lead to a central splitter, and central splitters work well. In the case of sorting, a central splitter divides the problem into two considerably smaller subproblems.

To simplify the presentation, we will slightly modify the algorithm so that it only issues its recursive calls when it finds a central splitter. Essentially, this modified algorithm differs from Quicksort in that it prefers to throw away an “off-center” splitter and try again; Quicksort, by contrast, launches the recursive calls even with an off-center splitter, and at least benefits from the work already done in splitting S . The point is that the expected running time of this modified algorithm can be analyzed very simply, by direct analogy with our analysis for median-finding. With a bit more work, a very similar but somewhat more involved analysis can also be done for the original Quicksort algorithm as well; however, we will not describe this analysis here.

```

Modified Quicksort( $S$ ):
    If  $|S| \leq 3$  then
        Sort  $S$ 
    
```

```

    Output the sorted list
Endif
Else
    While no central splitter has been found
        Choose a splitter  $a_i \in S$  uniformly at random
        For each element  $a_j$  of  $S$ 
            Put  $a_j$  in  $S^-$  if  $a_j < a_i$ 
            Put  $a_j$  in  $S^+$  if  $a_j > a_i$ 
        Endfor
        If  $|S^-| \geq |S|/4$  and  $|S^+| \geq |S|/4$  then
             $a_i$  is a central splitter
        Endif
    Endwhile
    Recursively call Quicksort( $S^-$ ) and Quicksort( $S^+$ )
    Output the sorted set  $S^-$ , then  $a_i$ , then the sorted set  $S^+$ 
Endif

```

Consider a subproblem for some set S . Each iteration of the **While** loop selects a possible splitter a_i and spends $O(|S|)$ time splitting the set and deciding if a_i is central. Earlier we argued that the number of iterations needed until we find a central splitter is at most 2. This gives us the following statement.

(13.19) *The expected running time for the algorithm on a set S , excluding the time spent on recursive calls, is $O(|S|)$.*

The algorithm is called recursively on multiple subproblems. We will group these subproblems by size. We'll say that the subproblem is of *type j* if the size of the set under consideration is at most $n(\frac{3}{4})^j$ but greater than $n(\frac{3}{4})^{j+1}$. By (13.19), the expected time spent on a subproblem of type j , excluding recursive calls, is $O(n(\frac{3}{4})^j)$. To bound the overall running time, we need to bound the number of subproblems for each type j . Splitting a type j subproblem via a central splitter creates two subproblems of higher type. So the subproblems of a given type j are disjoint. This gives us a bound on the number of subproblems.

(13.20) *The number of type j subproblems created by the algorithm is at most $(\frac{4}{3})^{j+1}$.*

There are at most $(\frac{4}{3})^{j+1}$ subproblems of type j , and the expected time spent on each is $O(n(\frac{3}{4})^j)$ by (13.19). Thus, by linearity of expectation, the expected time spent on subproblems of type j is $O(n)$. The number of different types is bounded by $\log_{\frac{4}{3}} n = O(\log n)$, which gives the desired bound.

(13.21) *The expected running time of Modified Quicksort is $O(n \log n)$.*

We considered this modified version of Quicksort to simplify the analysis. Coming back to the original Quicksort, our intuition suggests that the expected running time is no worse than in the modified algorithm, as accepting the noncentral splitters helps a bit with sorting, even if it does not help as much as when a central splitter is chosen. As mentioned earlier, one can in fact make this intuition precise, leading to an $O(n \log n)$ expected time bound for the original Quicksort algorithm; we will not go into the details of this here.

13.6 Hashing: A Randomized Implementation of Dictionaries

Randomization has also proved to be a powerful technique in the design of data structures. Here we discuss perhaps the most fundamental use of randomization in this setting, a technique called *hashing* that can be used to maintain a dynamically changing set of elements. In the next section, we will show how an application of this technique yields a very simple algorithm for a problem that we saw in Chapter 5—the problem of finding the closest pair of points in the plane.



The Problem

One of the most basic applications of data structures is to simply maintain a set of elements that changes over time. For example, such applications could include a large company maintaining the set of its current employees and contractors, a news indexing service recording the first paragraphs of news articles it has seen coming across the newswire, or a search algorithm keeping track of the small part of an exponentially large search space that it has already explored.

In all these examples, there is a *universe* U of possible elements that is extremely large: the set of all possible people, all possible paragraphs (say, up to some character length limit), or all possible solutions to a computationally hard problem. The data structure is trying to keep track of a set $S \subseteq U$ whose size is generally a negligible fraction of U , and the goal is to be able to insert and delete elements from S and quickly determine whether a given element belongs to S .

We will call a data structure that accomplishes this a *dictionary*. More precisely, a dictionary is a data structure that supports the following operations.

- **MakeDictionary.** This operation initializes a fresh dictionary that can maintain a subset S of U ; the dictionary starts out empty.
- **Insert(u)** adds element $u \in U$ to the set S . In many applications, there may be some additional information that we want to associate with u